Check for updates

Transactions on Emerging Telecommunications Technologies

WILEY

RESEARCH ARTICLE

SeqFedRPC: Sequential Federated Learning With Regularized Parameter Clustering

Peng Mao[1] | Tian Du[2] | Zhonghui Wu[3] | Dan Wang[3] | Han Xiao[1,4]

[1]Beijing Key Laboratory of MEMS Technology and Device Reliability for Industrial Internet, School of Intelligent Engineering and Automation, Beijing University of Posts and Telecommunications, Beijing, China | [2]School of Business Administration, Southwestern University of Finance and Economics, Chengdu, Sichuan, China | [3]China Mobile, Beijing, China | [4]The State Key Laboratory of Networking and Switching Technology, School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing, China

Correspondence: Tian Du (dtian.nora@gmail.com)

Received: 15 May 2026 | **Revised:** 11 June 2026 | **Accepted:** 13 July 2026

Keywords: parameter decoupling | regularized parameter clustering | sequential federated learning

ABSTRACT

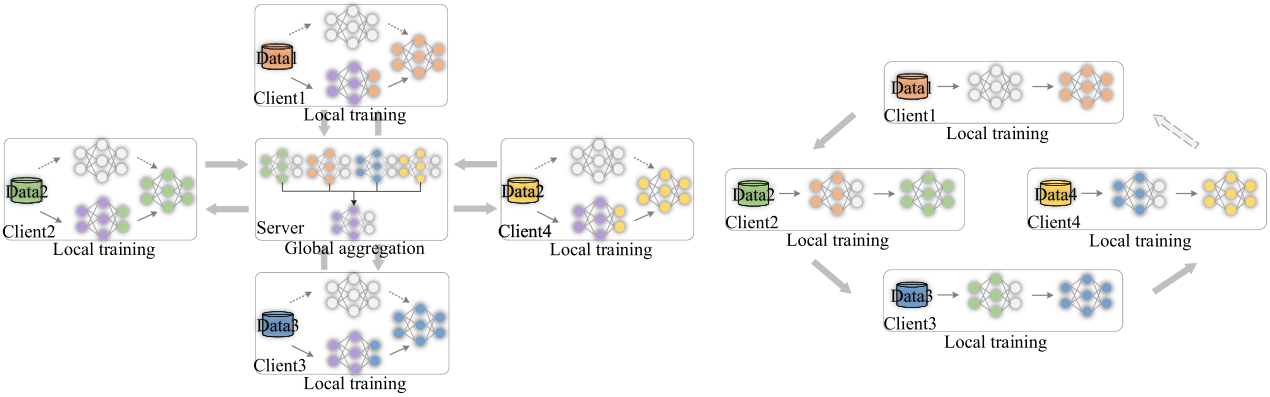
Sequential federated learning (SFL) enables collaborative model training across clients in a chain manner, providing communication-efficient benefits over traditional all-gather parallel federated learning (PFL). However, SFL training often suffers from slow convergence and performance degradation due to nonidentically distributed (non-IID) data distribution. In motivation experiment, we find that model decoupling by partitioning the model into shared and personalized parameters and using just a few personalized parameters with large gradients can improve SFL training performance. Based on above findings, we propose `SeqFedRPC`, a novel model decoupling based SFL framework with regularized parameter clustering. We introduce a regularization term to promote parameter sparsification and amplify gradient differences, which aids in gradient-based parameter clustering. Then, we employ a clustering-based scheme to adaptively decouple the model parameters into shared and personalized subsets, thereby addressing the challenge of non-IID data by adapting global knowledge with shared parameters and client-specific distributions with personalized parameters. Extensive experiments on eight benchmark datasets demonstrate that `SeqFedRPC` surpasses eight SOTA methods, with each client personalizing less than 10% of the total parameters on all datasets at $\alpha = 0.1$.

1 | Introduction

Federated Learning (FL) has emerged as a cutting-edge paradigm for training learning models collaboratively across multiple clients without the need to centralize data [1]. However, as scaling laws [2, 3] drive the trend toward larger models and datasets, FL faces significant challenges regarding the communication overhead associated with aggregating large-scale model parameters across numerous clients [4]. Sequential FL, as a communication-efficient distribution learning paradigm [5], has attracted increasing attention, such as ring-all-reduce based algorithms [6–8]. However, SFL suffers from low convergence and performance degradation by handling the inherent data heterogeneity among clients with non-independent and identically distributed (non-IID) data distributions [5].

To address data heterogeneity issues, personalized FL have been developed [9] for traditional parallel FL (PFL) and follows several routes. For example, knowledge distillation based methods [10, 11] focus on pruning and distilling smaller models, then aggregating only the distilled models. Similarity-based aggregation solutions [12, 13] group clients with similar data distributions and adjust the aggregation weights based on the clients' similarity. Model decoupling based approaches [14–16] separate models

© 2026 John Wiley & Sons Ltd.
All rights reserved, including rights for text and data mining and training of artificial intelligence technologies or similar technologies.



(a) Model decoupling in parallel federated learning.

(b) Model decoupling in sequential federated learning.

FIGURE 1 | Training manner for PFL and SFL with model decoupling.

into shared and personalized layers. Clients retain personalized layers locally and only upload parameters of the shared layers for aggregation. Parameter decoupling [17, 18], as a finer-grained version of model decoupling, divides parameters of the model into shared and personalized subsets, aggregating only shared parameters. Parameter decoupling is a promising solution for improving performance in non-IID scenarios. However, parameter decoupling based methods in the SFL domain have yet to be explored. Applying parameter decoupling to SFL is not a straightforward extension of personalized PFL. In SFL, personalization is closely tied to adjacent-client interference: client-specific parameters learned from one local distribution may be propagated to subsequent clients with different distributions, thereby accumulating client drift along the training chain. Therefore, an SFL-oriented decoupling method should not only identify personalized parameters, but also control how client-specific parameters are inherited or protected during sequential model passing.

Figure 1 shows the training methods for PFL and SFL with model decoupling. In SFL, clients share parameters sequentially, while PFL employs all-gather. However, PFL incurs higher communication overhead due to full model distribution, whereas SFL employs ring-based model delivery, which reduces bandwidth resource overhead. We consider applying parameter decoupling in SFL training to address data heterogeneity challenges. However, a key issue is determining which model parameters to personalize in SFL training. The current parameter decoupling methods in PFL use a simple threshold-based binary split into personalized and shared parameters [17, 18]. A key shortcoming of these designs is their inability to determine which parameters need personalization and how many should be adapted. These considerations motivate exploring a new parameter decoupling framework for SFL. Therefore, we conduct experiments using three models including LeNet-5 [19], ResNet-18 [20], and VGG-11 [21] focused on identifying the impact of each parameter for SFL training performance, revealing two interesting insights.

- Selecting the parameters with large local gradients during SFL training between two adjoining clients as personalized enhances performance.
- The parameter updates follow a power-law distribution, and selecting only a small percentage of parameters for personalization can improve accuracy.

These results inspired a new approach that excludes a subset of parameters with large differences during the model update between two sequential clients in the SFL chain manner. This perspective has also been implicitly supported in some prior PFL studies, which suggest that parameters with low gradients tend to act as background noise [22], while high-magnitude gradients are more likely to represent personalization [18].

In this paper, we propose `SeqFedRPC`, a novel model-decoupling-based training framework for SFL. Unlike PSPFL [18], which identifies personalized parameters from local parameter updates using a predefined threshold, `SeqFedRPC` builds on regularized gradient differences and adaptive clustering. Specifically, the regularized loss amplifies parameter-update discrepancies, making parameters that are more sensitive to local data distributions more distinguishable from broadly shared ones. Based on the enhanced gradient-based scores, `SeqFedRPC` employs K-Means clustering to automatically determine the parameter decoupling boundary, avoiding manually specified thresholds or fixed personalization ratios. During the main training stage, `SeqFedRPC` further introduces a clustering-based parameter decoupling mechanism. Each client filters out personalized parameters from the model received from the previous client, selectively inherits the shared parameter subset, and merges it with its local model to obtain the initial model. The client then performs local training on this initialized model and passes the trained model to the next client in the SFL chain. In this way, each client updates its local model using shared parameters while preserving its personalized parameters unchanged. We summarize our main contributions as follows:

- *Experiment findings*: Our study reveals two key findings in the personalized parameter selection of SFL. First, the parameters with larger gradients are more suitable for personalization. Second, only a small population of the personalized parameters is needed for adapting to the client-specific data distribution.
- *Algorithm design*: Traditional threshold-based parameter decoupling needs a hyper-parameter is needed to determine the partitioning ratio. To overcome this, we introduce a novel framework called `SeqFedRPC` with two key contributions. First, we propose a new regularization term to amplify gradient differences to boost parameter clustering. Second, we

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

apply K-Means clustering to adaptively categorize the model parameters into shared and personalized groups.

- *Empirical evaluation*: To validate the effectiveness of SeqFedRPC, we conducted extensive experiments on eight benchmark datasets, along with ablation studies and hyperparameter analysis. The results demonstrate that our method outperforms 11 compared methods, including three classical baselines and eight state-of-the-art FL methods.

2 | Related Work

In this section, we review related studies from two aspects: FL with non-IID data distributions and SFL. For FL under non-IID data, we further organize prior studies from client-level clustering to model-level decoupling and parameter-level decoupling, which are closely related to the motivation of SeqFedRPC.

2.1 | Federated Learning With Non-IID Data Distributions

FL with non-IID data has been widely studied to alleviate client drift and performance degradation caused by heterogeneous local distributions. Existing studies can be organized into client clustering, model decoupling, and parameter decoupling methods. Client clustering methods aim to group clients with similar data distributions or training behaviors so that more suitable collaboration can be performed within each group. IFCA [12] groups clients by minimizing the local loss on cluster models. CPPer-FL [23] applies K-Means to cluster clients according to the similarity of model parameters, and clients within the same cluster share local models for aggregation. These methods reduce the negative influence of heterogeneous clients by identifying client-level similarities. However, they mainly determine which clients should collaborate, rather than which model parameters should be shared or personalized. Model decoupling methods address data heterogeneity by separating the model into shared and personalized components. For example, FedRep [14] and pFedSim [24] partition the model into a globally shared representation part and a locally personalized classifier or head. FedPAC [16] further introduces feature alignment with global knowledge to reduce the discrepancy among feature extractors across clients. However, most model decoupling methods adopt coarse-grained layer-wise separation, and such designs may overlook fine-grained heterogeneity among parameters within the same layer.

Parameter decoupling provides a finer-grained solution by dividing model parameters into shared and personalized subsets. For example, PSPFL [18] uses local parameter updates to identify important parameters and retains those above a predefined threshold as personalized parameters. FedDPA [17] employs Fisher information to evaluate parameter importance, keeping high-value parameters locally and sharing low-value parameters for global aggregation. However, existing parameter decoupling methods are mainly developed for PFL, where clients upload local updates to a central server for aggregation. Moreover, they often rely on manually specified thresholds or fixed partitioning rules, making it difficult to adaptively determine both which parameters should be personalized.

2.2 | Sequential Federated Learning

SFL has attracted increasing attention as a communication-efficient alternative to PFL [25]. Instead of aggregating models from all selected clients through an all-gather manner, SFL passes the model through clients in a chain-like order, which can reduce communication overhead. Existing SFL studies [5, 25–31] mainly focus on improving sequential training mechanisms, client ordering, or cluster-wise updating. For example, TornadoAggregate [26] utilizes a ring-based architecture and reformulates loss minimization as variance reduction to improve scalability. FedSeq [27] introduces "superclients" to enable sequential training over subgroups of clients with heterogeneous data. FedDC [29] periodically reallocates local models among clients through a privacy-preserving daisy-chain strategy. Fed-CHS [31] activates one cluster at a time for sequential updating, while other studies organize clients into groups for sequential updates within groups and parallel updates between groups.

SFL still suffers from slow convergence and performance degradation under non-IID data distributions, as sequential model propagation may accumulate client drift across heterogeneous clients. Meanwhile, existing parameter decoupling methods are mainly designed for PFL and are not tailored to the sequential model-passing paradigm, leaving adaptive identification of shared and personalized parameters in SFL underexplored. To fill this gap, we propose SeqFedRPC, a SFL framework with regularized parameter clustering. SeqFedRPC adaptively distinguishes shared and personalized parameters within SFL without client clustering and uses a regularized objective to enhance parameter-level separability for more reliable clustering-based decoupling under non-IID data.

3 | Motivation

This section introduces the motivation, experiment setups, and two key findings. We use three models, including LeNet-5 [19], VGG-11 [21], and ResNet-18 [20]. The number of parameters and neural network layers are presented in Table 1, while the architectural details are available in A. The experiments are conducted on the CIFAR10 [32] dataset. To simulate non-IID scenarios of SFL, the dataset was partitioned into 100 subsets using a Dirichlet distribution (Dir(α)) with $\alpha = 0.1$. More details are shown in Section 5.1.

Figure 2a displays the gradient distribution for the VGG-11 model on CIFAR-10, using 100 bins ranging from 0 to 0.08 on a logarithmic scale. Most gradient differences are near 0, showing small changes are common. As values rise, occurrences decrease,

TABLE 1 | Model architectures and parameter counts for LeNet-5, VGG-11, and ResNet-18.

Model	LeNet-5	VGG-11	ResNet-18
Total parameters	60 646	132 863 032	11 689 512
Conv/FC layers	2/3	8/3	17/1

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

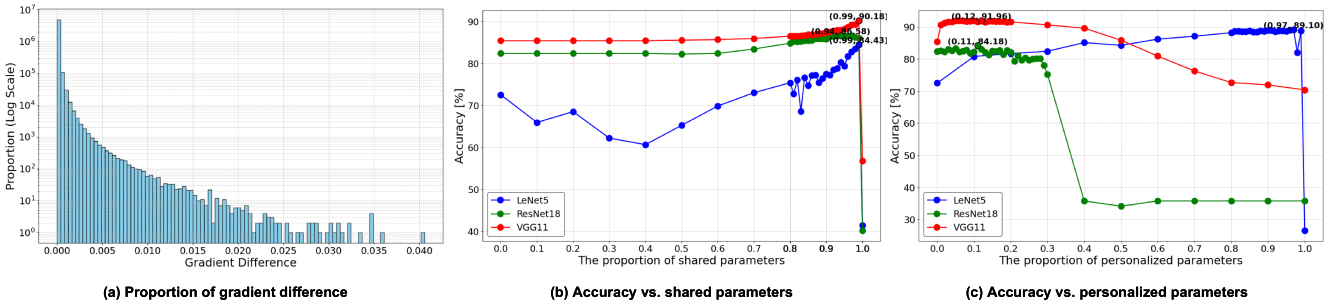


FIGURE 2 | Analysis of parameter characteristics with three model on CIFAR10. (a) Proportion of gradient differences; (b) Accuracy with varying proportions of shared parameters; (c) Accuracy with varying proportions of personalized parameters.

indicating fewer parameters with large gradient differences during SFL training. We compare the accuracy with different proportions of shared parameters on the ResNet-18 (green line), VGG-11 (red line), and LeNet-5 (blue line) models in Figure 2b. As the proportion of shared parameters increases from 0.0 to 1.0, LeNet-5's accuracy fluctuates but generally remains below 80%. Both ResNet-18 and VGG-11 demonstrate high accuracy, with VGG-11 and ResNet-18 stabilizing above 80%. All models achieve peak performance when shared parameters reach almost 95%–99%, but suffer a dramatic decline to 40%–60% at 100%. The above experiments reveal that selecting a small proportion of shared parameters diminishes collaborative training effectiveness in SFL. Figure 2c shows how accuracy changes with the proportion of personalized parameters. LeNet-5 exhibits relatively high accuracy starting from a low proportion of personalized parameters, peaking at around 97%. VGG-11 performs best with a 12% proportion. ResNet-18 drops below 40% accuracy after a 30% proportion.

The above results suggest that the parameters with larger updates are more suitable for personalization. Since few parameters have large gradients and selecting too many personalization parameters can degrade performance, only a small subset is needed for personalization.

4 | Method

In this section, we give the preliminaries of the parameter decoupling in PFL. Then, we introduce a novel parameter decoupling based framework called `SeqFedRPC` with regularized parameter clustering for personalized parameter selection. For notation, scalars are represented by lowercase italic letters. Vectors and sets are denoted by lowercase bold italics and uppercase italics, respectively. Matrices are indicated with uppercase, italic, and bold fonts. The symbol Δ represents the differential operator, while $|\cdot|$ and $\|\cdot\|$ are used for the 1-norm and 2-norm, respectively. Table 2 lists the mathematical notations used in this paper.

TABLE 2 | Mathematical notations.

Symbol	Description
T	The total communication rounds
$\mathcal{S}$	The set of the clients
$\mathrm{Dir}(\alpha)$	The Dirichlet distribution parameterized by α
β	The participation rate at each round
$m(t)$	The number of selected clients at time t
$\mathcal{D}, \mathcal{D}_s$	The total dataset and the local dataset of the s-th client
$\boldsymbol{\sigma}(t)$	The sequential training order at round t
$\boldsymbol{w}$	The total parameters of the model
$\mathcal{L}(\boldsymbol{w}; \mathcal{D})$	The loss function for the model $\boldsymbol{w}$ on dataset $\mathcal{D}$
$\boldsymbol{u}_{\sigma_i}(t)$	The shared subset of the client σ_i at round t
$\boldsymbol{v}_{\sigma_i}(t)$	The personalized subset of the client σ_i at round t
η	The learning rate for local training
λ	The weight of the L_1 regularization term
γ	The weight of the convex regularization term
τ	The threshold for clustering-based model decoupling
$\mathcal{K}$	The index set for all model parameters $\boldsymbol{w}$

4.1 | SFL With Parameter Decoupling

4.1.1 | Sequential Federated Learning

The SFL training process consists of T communication rounds. In each round t, the server selects a subset of clients $\mathcal{S}(t) \subset \mathcal{S}$, where $|\mathcal{S}(t)| = \beta \cdot |\mathcal{S}|$ represents the number of participating clients, with $t \in [1, T]$ and participation rate β. We set the total dataset as $\mathcal{D}$, and $\mathcal{D}_s$ represents the local dataset of client s, where $|\mathcal{D}|$ and $|\mathcal{D}_s|$ represent the size of the total dataset and the s-th local dataset, respectively. The standard PFL procedure begins with the server determining the sequential training order of the client indices $\boldsymbol{\sigma}(t) = \{\sigma_1, \sigma_2, \ldots, \sigma_{m(t)}\}$, which are randomly sampled without replacement from the set $\{1, 2, \ldots, |\mathcal{S}|\}$. Then, the server distributes an initial model $\boldsymbol{w}_g(0)$ to the first selected clients $\mathcal{S}(0)$. The client then performs local training using their local dataset $\mathcal{D}_s$. After the multi-round sequential training, the last client s in the order $\boldsymbol{\sigma}(t)$ returns the trained model $\boldsymbol{w}_s(t)$ to the server. The objective function $\mathcal{F}(\cdot)$ can be formulated as follows:

$$\arg\min_{\mathbf{w}} \left\{ \mathcal{F}\left(\{\mathbf{w}_s\}; \mathcal{D}\right) \coloneqq \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \mathcal{L}\left(\mathbf{w}_s; \mathcal{D}_s\right) \right\}, \tag{1}$$

where $\mathcal{L}\left(\mathbf{w}_s; \mathcal{D}_s\right) = \mathbb{E}_{(x,y)\sim\mathcal{D}_s}\left[\ell\left(\mathbf{w}_s; (x, y)\right)\right]$ represents the expected loss over the local dataset $\mathcal{D}_s$ of the client s, with $\ell(\cdot)$ denoting the loss function. Following local training, participating clients deliver their trained model parameters $\left\{\mathbf{w}_s(t)\right\}_{s\in\mathcal{S}(t)}$ to their next clients.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

4.1.2 | Parameter Decoupling

To handle Non-IID client data distributions, parameter decoupling solution splits model parameters $\mathbf{w}_s(t)$ into shared subset $\mathbf{u}_s(t)$ and personalized subset $\mathbf{v}_s(t)$ for each client s, where $\mathbf{w}_s(t) = \{\mathbf{u}_s(t), \mathbf{v}_s(t)\}$. This separation is based on gradient [18, 33] or Fisher values [17]. Therefore, we can rewrite Equation (1) as:

$$\arg\min_{\{\mathbf{u}_s,\mathbf{v}_s\}} \left\{ \mathcal{F}\left(\{\mathbf{u}_s,\mathbf{v}_s\};\mathcal{D}\right) \coloneqq \frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}} \mathcal{L}\left(\mathbf{u}_s,\mathbf{v}_s;\mathcal{D}_s\right) \right\}, \tag{2}$$

where $\mathcal{L}\left(\mathbf{u}_s,\mathbf{v}_s\right) = \mathbb{E}_{(x,y)\sim\mathcal{D}_s}\left[\ell\left(\{\mathbf{u}_s,\mathbf{v}_s\};(x,y)\right)\right]$ represents the expected loss for client s with its decoupled parameters.

The standard workflow of gradient-based SFL with parameter decoupling [18, 33] in one round t with parameter decoupling is as follows:

- `Model Merging`: For the first client σ_1 in order $\boldsymbol{\sigma}(t)$, it get the model parameters $\mathbf{w}_{\sigma_{m(t-1)}}$ from the last client $\sigma_{m(t-1)}$ of previous round $t-1$. For other condition, the client σ_i updates its local model $\mathbf{w}_{\sigma_i}(t)$ with the received parameters $\mathbf{u}_{\sigma_{i-1}}(t)$ from the previous client σ_{i-1} as follows:

$$\widehat{\mathbf{w}}_{\sigma_i}(t) = \left\{ \left\{\mathbf{w}_{\sigma_{i-1}}(t)\right\}_{\mathcal{K}(\mathbf{u}_{\sigma_i})},\ \mathbf{v}_{\sigma_i}(t-1) \right\}. \tag{3}$$

  We denote $\mathcal{K}\left(\mathbf{w}_{\sigma_i}(t)\right)$ as the parameter index set for $\mathbf{w}_{\sigma_i}(t)$, where $\mathbf{w}_{\sigma_i}(t) = \left\{ w^k_{\sigma_i}(t) \right\}_{k\in\mathcal{K}(\mathbf{w}_{\sigma_i}(t))}$ and $w^k_{\sigma_i}(t)$ is the parameter at position k.

- `Local Updating`: The client σ_i performs local training on datasets $\mathcal{D}_{\sigma_i}(t)$ and computes the local gradient $\Delta\widehat{w}_{\sigma_i}(t)$ as follows:

$$\Delta\widehat{\mathbf{w}}_{\sigma_i}(t) = \eta\cdot\nabla_{\widehat{\mathbf{w}}_{\sigma_i}}\mathcal{L}\left(\widehat{\mathbf{w}}_{\sigma_i}(t);\mathcal{D}_{\sigma_i}\right), \tag{4}$$

  where η is the learning rate and $\nabla_{\widehat{\mathbf{w}}_{\sigma_i}}\mathcal{L}\left(\widehat{\mathbf{w}}_{\sigma_i}(t);\mathcal{D}_{\sigma_i}\right)$ represents the gradient of the loss with respect to the parameters $\widehat{\mathbf{w}}_{\sigma_i}(t)$ on $\mathcal{D}_{\sigma_i}$.

- `Model Decoupling`: Each client σ_i decouples its local trained model $\mathbf{w}_{\sigma_i}(t)$ based on the gradient $\Delta\widehat{\mathbf{w}}_{\sigma_i}(t)$ as follows:

$$\mathcal{K}\left(\mathbf{u}_{\sigma_i}\right) = \left\{ w^k_{\sigma_i}(t) \right\}_{\Delta\widehat{w}^k_{\sigma_i}(t)<\tau}, \mathcal{K}\left(\mathbf{v}_{\sigma_i}\right) = \left\{ w^k_{\sigma_i}(t) \right\}_{\Delta\widehat{w}^k_{\sigma_i}(t)\geq\tau}, \tag{5}$$

  where τ is the quantiling threshold. Then, the client sends $\mathbf{u}_{\sigma_i}(t)$ to the next client σ_{i+1}.

4.2 | `SeqFedRPC` Framework

4.2.1 | Regularized Loss Function

`SeqFedRPC` incorporates a new regularization term into the local loss function $\mathcal{L}\left(\mathbf{w}_s\right)$ for the parameter clustering to facilitate the model decoupling. Formally, we define the regularized loss function $\mathcal{L}^{\dagger}\left(\mathbf{w}_s\right)$ for each client s as follows:

$$\mathcal{L}^{\dagger}\left(\mathbf{w}_s\right) = \mathcal{L}\left(\mathbf{w}_s\right) + \lambda\parallel\mathbf{w}_s\parallel + \gamma f\left(\parallel\mathbf{w}_s-\mathbf{w}\parallel\right), \tag{6}$$

where $\lambda\parallel\mathbf{w}_s\parallel$ is the L_1 regularization term promoting sparsity in the model parameters. $\gamma f\left(\parallel\mathbf{w}_s-\mathbf{w}\parallel\right)$ is the convex regularization term designed to scale the parameter differences, where $f:\mathbb{R}_{\geq 0}\to\mathbb{R}$ be a differentiable, convex, and nondecreasing function, $\lambda,\gamma>0$.

Proof. Based on Equation (6), the gradient of $\mathcal{L}^{\dagger}\left(\mathbf{w}_s\right)$ with respect to $\mathbf{w}_s$ can be express as:

$$\nabla_{\mathbf{w}_s}\mathcal{L}^{\dagger}\left(\mathbf{w}_s\right) = \nabla_{\mathbf{w}_s}\mathcal{L}_s\left(\mathbf{w}_s\right) + \lambda\nabla_{\mathbf{w}_s}\parallel\mathbf{w}_s\parallel + \gamma\nabla_{\mathbf{w}_s} f\left(\parallel\mathbf{w}_s-\mathbf{w}\parallel\right).$$

Let $d \coloneqq \parallel\mathbf{w}_s-\mathbf{w}\parallel$. The gradient of the regularization term with respect to $\mathbf{w}_s$ is given by:

$$\gamma\nabla_{\mathbf{w}_s} f\left(\parallel\mathbf{w}_s-\mathbf{w}\parallel\right) = \nabla_{\mathbf{w}_s}[\gamma f(d)] = \gamma f'(d)\cdot\frac{\mathbf{w}_s-\mathbf{w}}{d}$$

Thus, the gradient norm is:

$$\left\|\nabla_{\mathbf{w}_s}[\gamma f(d)]\right\| = \gamma\mid f'(d)\mid = \gamma f'(d)$$

where the last equality follows from $f'(d)\geq 0$, since f is nondecreasing. Moreover, the convexity of f implies that f' is a nondecreasing function of r. Therefore, as $d = \parallel\mathbf{w}_s-\mathbf{w}\parallel$ increases, the gradient norm increases as well. □

In SFL, the model is updated sequentially by clients with heterogeneous local objectives. When a client's data distribution differs from the global one, its update may be biased toward the local optimum. As the updated model is passed to subsequent clients, such bias can accumulate along the training chain, causing client drift and unstable optimization. The proposed convex regularization term addresses this issue through a distance-aware correction with respect to the reference parameter $\mathbf{w}$. When the local parameter $\mathbf{w}_s$ is close to $\mathbf{w}$, the regularization remains mild, allowing effective local adaptation. As $\mathbf{w}_s$ moves farther from $\mathbf{w}$, the convex and nondecreasing regularizer induces a stronger gradient, thereby penalizing excessive local deviation. This adaptive correction stabilizes the trajectory of shared parameters and mitigates the accumulation of client-specific bias along the SFL chain.

The regularization term further facilitates parameter decoupling. Parameters consistently optimized across clients tend to stay close to the reference model and produce smaller gradient-based scores, whereas parameters sensitive to client-specific non-IID data exhibit larger deviations and update scores. By enlarging the separability between these two groups, the proposed regularization enables K-Means to more reliably distinguish shared from personalized parameters. Consequently, drift-prone parameters are retained locally, while more stable ones are propagated along the client chain, improving personalization and convergence under non-IID data.

4.2.2 | Clustering-Based Parameter Decoupling

The `SeqFedRPC` framework categorizes model parameters into shared and personalized groups through a clustering-based quantiling solution as shown in Figure 3. Different from the standard workflow of SFL with model decoupling, the `SeqFedRPC` employs clustering-based model decoupling and local training with the regularized loss function. Since the `model merging` is

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

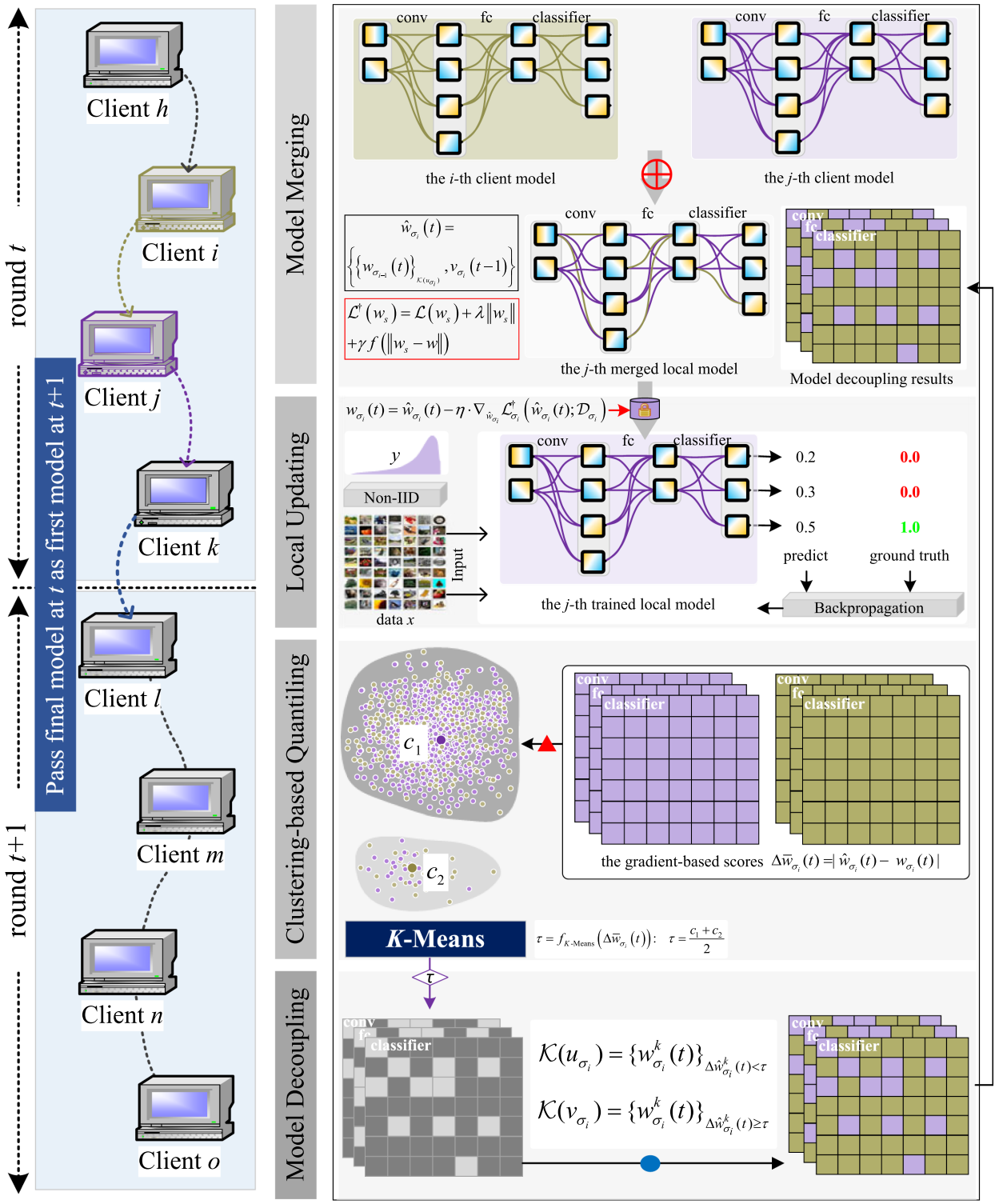


FIGURE 3 | The SeqFedRPC framework with clustering-based parameter decoupling.

processed consistently with standard SFL, we omit this step here. The main training stage implementation, with `model decoupling` and `local training`, is as follows:

- `Local Training`: The client σ_i performs local training on datasets $\mathcal{D}_{\sigma_i}(t)$ and obtain the trained model $\mathbf{w}_{\sigma_i}(t)$ as follows:

$$\mathbf{w}_{\sigma_i}(t) = \hat{\mathbf{w}}_{\sigma_i}(t) - \eta \cdot \nabla_{\hat{\mathbf{w}}_{\sigma_i}} \mathcal{L}^{\dagger}_{\sigma_i}\left(\hat{\mathbf{w}}_{\sigma_i}(t); \mathcal{D}_{\sigma_i}\right). \tag{7}$$

We denote $\Delta\overline{\mathbf{w}}_{\sigma_i}(t) = |\,\hat{\mathbf{w}}_{\sigma_i}(t) - \mathbf{w}_{\sigma_i}(t)\,|$ as the gradient-based scores.

- `Clustering-based Quantiling`: The client σ_i applies K-Means clustering to the one-dimensional gradient scores $\Delta\overline{\mathbf{w}}_{\sigma_i}(t)$ for parameter decoupling. The scores are partitioned into two clusters representing shared and personalized parameters, respectively, with $K = 2$. Let c_1 and c_2

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ALGORITHM 1 | SeqFedRPC (T, β).

Input: Total rounds T, participation rate β.
Output: Trained models $\{\mathbf{w}_s(T)\}$.

for round $t = 1$ to T **do:**
Select subset of clients $S(t) \subset S$ with $| S(t) | = \beta \cdot | S |$;
Determine the order $\boldsymbol{\sigma}(t) = \{\sigma_1, \sigma_2, \ldots, \sigma_{m(t)}\}$ for $S(t)$;

for in sequence $\boldsymbol{\sigma}(t) = \{\sigma_1, \sigma_2, \ldots, \sigma_{m(t)}\}$ **do:**
if $i = 1$ **then:**
Set $\mathbf{w}_{\sigma_i}(t) \leftarrow \left\{ \left\{ \mathbf{w}^k_{\sigma_{m(t-1)}}(t) \right\}_{k \in \mathcal{K}(\mathbf{u}_{\sigma_i})}, \ \mathbf{v}_{\sigma_i}(t-1) \right\}$;
else:
Set $\mathbf{w}_{\sigma_i}(t) \leftarrow \left\{ \left\{ \mathbf{w}^k_{\sigma_{i-1}}(t) \right\}_{k \in \mathcal{K}(\mathbf{u}_{\sigma_i})}, \ \mathbf{v}_{\sigma_i}(t-1) \right\}$;
Update the model $\mathbf{w}_{\sigma_i}(t)$ with local datasets $\mathcal{D}_{\sigma_i}$ as:

$$\mathbf{w}_{\sigma_i}(t) = \hat{\mathbf{w}}_{\sigma_i}(t) - \eta \cdot \nabla_{\hat{\mathbf{w}}_{\sigma_i}} \mathcal{L}^{\dagger}_{\sigma_i}\left(\hat{\mathbf{w}}_{\sigma_i}(t); \mathcal{D}_{\sigma_i}\right).$$

Apply K-Means clustering on $\Delta\mathbf{w}_{\sigma_i}$ to determine the quantiling threshold τ;
Categorize parameters into $\mathcal{K}(\mathbf{u}_{\sigma_i})$ and $\mathcal{K}(\mathbf{v}_{\sigma_i})$ based on τ;
Deliver the full trained model $\mathbf{w}_{\sigma_i}(t)$ to the next client σ_{i+1}.
Return $\{\mathbf{w}_s(T)\}_{s \in S}$

be the two cluster centers after fitting. For eaxmples, we can take the midpoint $\tau = \frac{c_1 + c_2}{2}$ as the threshold. Thus, the K-Means-based threshold function is defined as:

$$\tau = f_{\text{K-Means}}\left(\Delta\overline{\mathbf{w}}_{\sigma_i}(t)\right) : \quad \tau = \frac{c_1 + c_2}{2}. \tag{8}$$

The client σ_i can separate parameters into high-importance group $\left\{\Delta\overline{w}^k_{\sigma_i}(t) \geq \tau\right\}$ and low-importance group $\left\{\Delta\widehat{w}^k_{\sigma_i}(t) < \tau\right\}$.

- `Model Decoupling`: Each client σ_i decouples its local trained model $\mathbf{w}_{\sigma_i}(t)$ based on the gradient scores $\Delta\overline{\mathbf{w}}_{\sigma_i}(t)$ as follows:

$$\mathcal{K}(\mathbf{u}_{\sigma_i}) = \left\{ w^k_{\sigma_i}(t) \right\}_{\Delta\overline{w}^k_{\sigma_i}(t) < \tau}, \mathcal{K}(\mathbf{v}_{\sigma_i}) = \left\{ w^k_{\sigma_i}(t) \right\}_{\Delta\overline{w}^k_{\sigma_i}(t) \geq \tau}, \tag{9}$$

where τ is the quantiling threshold and $w^k_{\sigma_i}(t)$ represents the parameter at position k. Then, the client σ_i sends all parameters $\mathbf{w}_{\sigma_i}(t)$ to the next client σ_{i+1}.

4.3 | Algorithm Design

We provide the main steps of `SeqFedRPC` algorithm as shown in Algorithm 1. Over T communication rounds, the server selects a subset of clients $S(t)$ with a participation rate β at each round and determines the ordering of the selected clients randomly. The selected clients perform local training followed by regularized loss $\mathcal{L}^{\dagger}(\cdot)$ in the order $\boldsymbol{\sigma}(t)$. Each client σ_i receives the model $\mathbf{w}_{\sigma_{i-1}}(t)$ from the previous client σ_{i-1} and merges the models $\mathbf{w}_{\sigma_{i-1}}(t)$ and $\mathbf{w}_{\sigma_i}(t-1)$ based on decoupling results $\mathcal{K}(\mathbf{u}_{\sigma_i})$ and $\mathcal{K}(\mathbf{v}_{\sigma_i})$. After local training, each client obtains the gradient-based scores $\Delta\hat{\mathbf{w}}_{\sigma_i}(t)$ and employs K-Means to determine the quantile threshold τ. Then, each client decouples the model into the shared subset $\mathcal{K}(\mathbf{u}_{\sigma_i})$ and the personalized subset $\mathcal{K}(\mathbf{v}_{\sigma_i})$ based on τ and share all parameters of the trained model $\mathbf{w}_{\sigma_i}(t)$ to the next client σ_{i+1}.

4.3.1 | Implementation and Complexity Analysis

The main training stage involves T communication rounds, yielding a per-round complexity of $\mathcal{O}\left(T \cdot \left(\beta \cdot |S| \cdot \left(E \cdot |\mathcal{D}_s| + |\mathcal{K}|\right)\right)\right)$. The number of the selected clients is $\beta \cdot | S |$, where they perform local updates and selective gradient uploads. Including server-side operations, the complexity of our `SeqFedRPC` algorithm scales linearly with the communication rounds T and the participating clients $\beta \cdot | S |$. It also depends on the sample size of the local dataset $\mathcal{D}_s$ and the model parameters scale $| \mathcal{K} |$. Our approach is linearly efficient as the system scales.

5 | Experiments

This section presents the experimental setup, datasets, compared methods, and parameter configurations. To evaluate the performance of the proposed algorithm `SeqFedRPC`, comprehensive comparisons are conducted against eight SOTA methods across eight benchmark datasets. The evaluation focuses on multiple aspects, including model accuracy, convergence analysis, hyperparameter sensitivity, ablation studies, and communication overhead.

5.1 | Experiment Setup, Datasets, and Baselines

All experiments are conducted using the open-source federated learning benchmark framework FL-Bench at https://github.com/KarhouTam/FL-bench. The experimental setup is deployed on a high-performance server equipped with an NVIDIA GeForce RTX 4090 GPU (24GB) and an Intel i9-13900K CPU (24 cores, 3.00 GHz). The source code and implementation details of the proposed method are available in the associated open-source repository https://github.com/tian0920/SeqFedRPC.

5.1.1 | Experimental Setup

All methods are implemented based on the VGG-11 [21] model architecture and evaluated across eight benchmark datasets. The federated learning setup involves 100 clients and a total of $T = 100$ communication rounds, with a client participation rate of $\beta = 0.1$ in each round. All edge clients adopt Stochastic Gradient Descent (SGD) as the optimizer, using a learning rate of 0.001. Each client performs $E = 5$ local training epochs per round with a batch size of 32.

5.1.2 | Datasets

Our method was evaluated on eight benchmark datasets including CIFAR10, CIFAR100, CINIC10, EMNIST, FMNIST, MNIST, SVHN and Tiny ImageNet (T.IN.), as detailed in Table 3. Each dataset is evenly split into nonoverlapping train and test sets per client, with heterogeneity simulated using Dir(α), $\alpha \in$

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 3 | Statistical information of used datasets on clients.

Dataset	Samples	Classes	Description	Resolution	Year
CIFAR10/100	60 000	10/100	Images in 10/100 classes including airplanes, cars, birds, etc.	32×32	2009
CINIC10	270 000	10	Images in 10 classes including airplanes, cars, birds, etc.	32×32	2018
EMNIST	805 263	62	Extended MNIST with letters and digits	28×28	2017
FMNIST	70 000	10	Fashion item images	28×28	2017
MNIST	70 000	10	Handwritten digits	28×28	1998
SVHN	600 000	10	Street view house numbers	32×32	2011
Tiny ImageNet	110 000	200	Tiny ImageNet dataset	64×64	2015

{0.1,0.5,1.0} distribution (lower values indicating higher heterogeneity). For example, when $\alpha = 0.1$, the data distributions across clients are highly imbalanced, and most edge clients may possess data samples from only a few classes. That is, $\mathcal{Y}_i \subset \mathcal{Y}$, where $\mathcal{Y}_i$ denotes the label space of the data samples held by client i, and $\mathcal{Y}$ represents the total label space.

5.1.3 | Compared Methods

To comprehensively evaluate the proposed method in terms of both generalization—under parallel and sequential patterns—and personalization performance, we compare it against three classical baselines: FedAvg [34], SFL, and Local-Only (where training is performed solely on local data without participating in global aggregation). Additionally, we benchmark against eight SOTA algorithms, categorized into four groups: (1) Clustering-based method: CFL [35]; (2) Regularization-based methods: FedDyn [36] and FedFomo [37]; (3) Model decoupling methods: FedPer [38], FedRep [14], LG-FedAvg [39], and pFedSim [24]; (4) Parameter decoupling method: PSPFL [18].

- `CFL` [35] evaluates the similarity between clients by capturing the geometric properties of gradient updates and dynamically clusters clients with similar characteristics into the same group.
- `FedDyn` [36] introduces a dynamic regularizer from the communication perspective, which dynamically adjusts the regularization parameters based on the results of each communication round and aligns the solutions of each edge client and server.
- `FedFomo` [37] quantitatively calculates the gain that each edge client can obtain from other clients during the parameter sharing, deriving the optimally weighted model combination for each client.
- `FedPer` [38] employs the last-layer-as-head decoupling by splitting the deep neural network into the base and personalized layers to counteract the adverse effects of statistical heterogeneity.
- `FedRep` [14] proposes a hybrid model with a globally shared representation and a locally personalized representation, where the classifier is treated as the personalized layer, trained locally, while the feature extractor is optimized sequentially.
- `LG-FedAvg` [39] adopts an alternative approach by training the feature extractor locally while training the classifier globally to alleviate the effects of data heterogeneity.
- `pFedSim` [24] based on Body-Head decoupling, the similarity between edge clients is computed using classifier parameters and used as weights for the global aggregation of the body layer.
- `PSPFL` [18] adopts a threshold-based approach to select important parameters by evaluating the changes of model parameters. The selected important parameters are uploaded to the server for aggregation, while the remaining parameters are retained and updated locally on the clients.

The key hyperparameter settings for the above baseline methods are listed below. All configurations are kept consistent with those reported in the original papers.

- `FedDyn` [36] sets $\alpha = 0.01$.
- `FedFomo` [37] sets the $M = 5$ and the ratio of validation set to 0.2.
- `FedRep` [14] sets the epoch for training the feature extractor part to 1.
- `LG-FedAvg` [39] sets the number of global layers to 1, with the remaining layers designated as local layers.
- `pFedSim` [24] sets the generalization ratio $\rho_{general} = 0.5$.

5.2 | Accuracy Comparison

Tables 4–6 present a comparison of the average model accuracy achieved by the proposed `SeqFedRPC` method against 11 other approaches, including baseline methods such as `FedAvg`, `SFL`, and `Local-Only`. The coefficient of the convex regularization term γ is set to 0.01, while the coefficient of the L_1 regularization term is set to 0. Under a highly heterogeneous setting ($\alpha = 0.1$), the proposed method generally achieves strong performance across datasets, obtaining the best or second-best results in most cases, with only marginal differences when not ranked first. The performance improvement is particularly evident on complex datasets. For example, `SeqFedRPC` achieves an accuracy of 60.488% on CIFAR100, outperforming the second-best method `FedPer` by 2.719%; on Tiny ImageNet, it outperforms `FedPer` by 0.436%. For moderately complex datasets such as CINIC10 and SVHN, `SeqFedRPC` also maintains its superiority, achieving

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 4 | Average model accuracy over datasets for $\alpha = 0.1$.

Method	CIFAR10	CIFAR100	CINIC10	SVHN	EMNIST	FMNIST	MNIST	T.IN.
FedAvg	54.983	30.382	45.894	72.488	80.448	76.434	94.648	35.999
Local-only	85.523	45.269	86.500	86.071	93.582	94.376	96.264	51.551
SFL	55.557	29.983	45.701	65.842	79.591	76.724	94.779	36.063
CFL	56.898	30.273	49.973	76.045	80.513	78.392	94.922	36.062
FedDyn	50.926	19.877	45.843	63.489	79.159	75.757	94.167	28.071
FedFomo	84.750	43.067	86.353	79.811	93.009	94.093	96.140	49.641
FedPer	**90.115**	57.769	87.921	94.107	**95.693**	95.105	**98.697**	59.906
FedRep	88.771	52.179	88.169	89.743	95.315	94.396	97.115	56.509
LG-FedAvg	86.625	45.123	86.646	84.494	93.597	94.131	96.460	51.686
pFedSim	85.839	45.732	84.366	87.084	93.970	93.643	96.526	51.629
PSPFL	84.693	40.235	84.915	84.679	93.567	94.300	96.742	51.559
SeqFedRPC	89.528	**60.488**	**88.885**	**94.562**	95.513	**95.896**	98.673	**60.342**
	↓ 0.587	↑**2.719**	↑**0.716**	↑**0.455**	↓ 0.180	↑**0.791**	↓ 0.024	↑**0.436**

Note: Optimal solutions are highlighted in bold, and second-best results in underline.

TABLE 5 | Average model accuracy over datasets for $\alpha = 0.5$.

Method	CIFAR10	CIFAR100	CINIC10	SVHN	EMNIST	FMNIST	MNIST	T.IN.
FedAvg	56.815	34.124	58.288	72.632	83.688	83.899	96.536	40.233
Local-only	60.759	20.538	66.434	64.108	83.943	83.771	91.308	24.327
SFL	62.915	23.828	50.995	72.715	80.777	77.144	91.982	40.207
CFL	56.925	33.722	58.735	81.167	83.776	83.895	96.626	40.230
FedDyn	59.622	24.316	53.943	67.078	83.103	81.955	95.906	34.915
FedFomo	60.401	19.063	66.113	57.315	83.823	83.588	91.824	20.795
FedPer	71.216	41.502	74.173	87.735	**90.617**	90.525	97.843	45.113
FedRep	72.788	31.393	74.094	82.145	89.790	87.490	95.494	35.178
LG-FedAvg	56.236	20.402	66.453	56.927	84.206	83.894	91.556	24.473
pFedSim	59.941	39.813	68.109	84.696	88.540	88.285	96.988	45.169
PSPFL	56.219	21.076	67.225	57.741	83.783	83.562	91.374	39.542
SeqFedRPC	**78.245**	**43.889**	**74.838**	**88.469**	90.156	**91.483**	**98.104**	**46.140**
	↑**5.457**	↑**2.387**	↑**0.665**	↑**0.734**	↓ 0.461	↑**0.958**	↑**0.261**	↑**0.971**

Note: Optimal solutions are highlighted in bold, and second-best results in underline.

TABLE 6 | Average model accuracy over datasets for $\alpha = 1.0$.

Method	CIFAR10	CIFAR100	CINIC10	SVHN	EMNIST	FMNIST	MNIST	T.IN.
FedAvg	59.837	34.308	53.905	75.097	81.667	78.976	93.673	40.717
Local-only	53.372	13.217	57.824	57.580	78.264	80.166	88.528	16.485
SFL	68.132	34.677	53.929	75.081	81.668	78.988	93.692	40.684
CFL	60.102	34.794	54.296	75.385	81.722	79.221	93.631	40.650
FedDyn	61.902	25.002	56.414	69.699	82.105	83.641	96.756	36.238
FedFomo	53.078	14.554	56.818	59.311	78.172	80.226	93.690	12.101
FedPer	67.347	36.077	66.383	81.836	86.168	84.966	95.221	40.836
FedRep	69.043	24.854	69.256	80.734	85.634	84.663	95.558	28.247
LG-FedAvg	52.187	13.026	57.569	49.599	78.187	79.970	88.691	16.683
pFedSim	70.919	35.851	66.964	83.533	**87.240**	87.214	97.221	42.251
PSPFL	55.021	18.067	59.692	60.976	79.237	79.941	88.433	34.805
SeqFedRPC	**72.704**	**39.184**	**70.525**	**87.617**	87.162	**88.418**	**97.580**	**44.010**
	↑**1.785**	↑**3.107**	↑**1.269**	↑**4.084**	↓ 0.078	↑**1.204**	↑**0.359**	↑**1.759**

Note: Optimal solutions are highlighted in bold, and second-best results in underline.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

accuracies of 88.885% and 94.562%, with improvements of 0.716% and 0.455% over FedRep and FedPer, respectively.

Table 5 presents the classification accuracy of various methods across eight datasets under a moderate data heterogeneity setting with $\alpha = 0.5$. Compared to the setting with $\alpha = 0.1$, this configuration represents a less heterogeneous data distribution. Despite the reduced heterogeneity, SeqFedRPC continues to demonstrate superior performance on most datasets, with the sole exception being EMNIST, where it ranks second, trailing the best result by only 0.461%. The performance gains are particularly pronounced on CIFAR10 and CIFAR100, where SeqFedRPC achieves 78.245% and 43.889% accuracy, outperforming the second-best methods by 5.457% and 2.387%, respectively. On CINIC10, SeqFedRPC attains an accuracy of 74.838%, improving upon FedRep by 0.665%. Similar performance gains are observed on the MNIST family of datasets: SeqFedRPC achieves 91.483% and 98.104% accuracy on FMNIST and MNIST, respectively, surpassing FedPer by 0.958% and 0.261%.

Table 6 presents the performance comparison of various methods across different datasets under the heterogeneity parameter setting of $\alpha = 1.0$. In this lowest-heterogeneity scenario, SeqFedRPC consistently maintains stable and superior performance across all datasets, with its advantages over other methods becoming more pronounced. The only exception is the EMNIST dataset, where SeqFedRPC ranks second, but the gap to the best-performing method is merely 0.078%, indicating a negligible difference. Notably, on the CIFAR100 and SVHN datasets, SeqFedRPC surpasses the second-best method by absolute margins of 3.107% and 4.084%, respectively. On moderately complex datasets such as CIFAR10, CINIC10, and FMNIST, SeqFedRPC also demonstrates competitive performance, achieving accuracies of 72.704%, 70.525%, and 88.418%, which exceed those of pFedSim and FedPer by 1.785%, 1.269%, and 1.204%, respectively.

Furthermore, by synthesizing the results from Tables 4–6, it can be observed that as the data distribution becomes more homogeneous, global aggregation-based federated learning methods such as FedAvg and SFL tend to perform increasingly well. This can be attributed to the enhanced consistency of client data under homogeneous conditions, which improves the representativeness and convergence of global updates. Specifically, at $\alpha = 1.0$, FedAvg and SFL achieve relatively strong performance on complex datasets such as CIFAR100 and Tiny ImageNet. In contrast, personalized federated learning methods such as FedPer, pFedSim, and LG-FedAvg tend to suffer performance degradation as their personalization advantages diminish in more uniform data distributions.

5.3 | Convergence Analysis

Figures 4–6 present convergence comparisons between SeqFedRPC and 11 other algorithms across eight datasets using the VGG-11 architecture. Each subplot illustrates the classification accuracy over 100 communication rounds, reflecting the convergence behavior of each method. To enhance readability, the curve of the proposed SeqFedRPC is highlighted with a bold red line. Figure 4 shows the convergence trajectories under the heterogeneity setting of $\alpha = 0.1$. The results demonstrate that SeqFedRPC consistently achieves the fastest convergence across all datasets. For example, on the CIFAR10 dataset, SeqFedRPC converges within approximately 40 rounds, while on more complex datasets such as CIFAR100 and Tiny ImageNet, convergence is reached in about 60 rounds. On SVHN and CINIC10, convergence is achieved in as few as 20–30 rounds, significantly outperforming all baseline methods.

Figure 5 illustrates the convergence trends of 11 methods across eight datasets under the heterogeneity setting of $\alpha = 0.5$. SeqFedRPC consistently demonstrates a significant advantage in convergence across most datasets. For example, on CIFAR10/100, FMNIST and MNIST, the proposed method not only achieves the fastest convergence speed but also reaches the highest classification accuracy. In contrast, methods such as FedFomo, FedDyn, and LG-FedAvg exhibit more pronounced fluctuations in their training curves, slower convergence, and inferior final performance. It is worth noting that although SeqFedRPC still converges faster on the EMNIST dataset, its average performance on this dataset is relatively lower compared to other methods.

Figure 6 shows the convergence behavior of 12 methods on eight datasets under a fully homogeneous setting ($\alpha = 1.0$). For simpler datasets like FMNIST, most methods converge within about 25 communication rounds. For moderately complex datasets such as CIFAR10 and CINIC10, convergence is typically reached within 40–50 rounds. SeqFedRPC consistently demonstrates strong convergence performance across all datasets, with fast convergence, stable training curves, and high accuracy. Moreover, from Figures 4–6, model decoupling methods (e.g., FedPer and FedRep) generally converge faster than traditional methods (e.g., FedAvg and SFL), highlighting the efficiency of parameter decoupling in improving training performance.

5.4 | Ablation Experiment

Figure 7 and Table 7 present the results of the ablation study, comparing the performance of our proposed method and several of its variants: (1) the SeqFedRPC framework, with the loss function hyperparameters set to $\lambda = 0$ and $\gamma = 0.01$; (2) SeqFedRPC_L_0, which adopts the standard cross-entropy loss function with $\lambda = 0$ and $\gamma = 0$; (3) SeqFedRPC_L_1, where we set $\lambda = 0.001$ and $\gamma = 0$; (4) SeqFedRPC_L_2, where we set $\lambda = 0.001$ and $\gamma = 0.01$; and (5) three baseline methods representing generalization, sequential adaptation, and full personalization, namely FedAvg, SFL, and Local-only, respectively. All experiments are conducted under the same setting: heterogeneity parameter $\alpha = 0.1$, client participation rate $\beta = 0.1$, local learning rate of 0.001, and a total of 100 communication rounds.

As shown in Figure 7 and Table 7, incorporating regularization terms into the loss function (i.e., SeqFedRPC_L_1, SeqFedRPC_L_2, and SeqFedRPC) generally achieves better performance than using only the standard cross-entropy loss (as in SeqFedRPC_L_0). Specifically, introducing only the L_1 regularization term ($\lambda = 0.001$, SeqFedRPC_L_1) yields the best results on datasets such as CIFAR10 and CINIC10; while applying only the convex regularization term ($\gamma = 0.01$, SeqFedRPC) performs particularly well on CIFAR100, SVHN, and

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

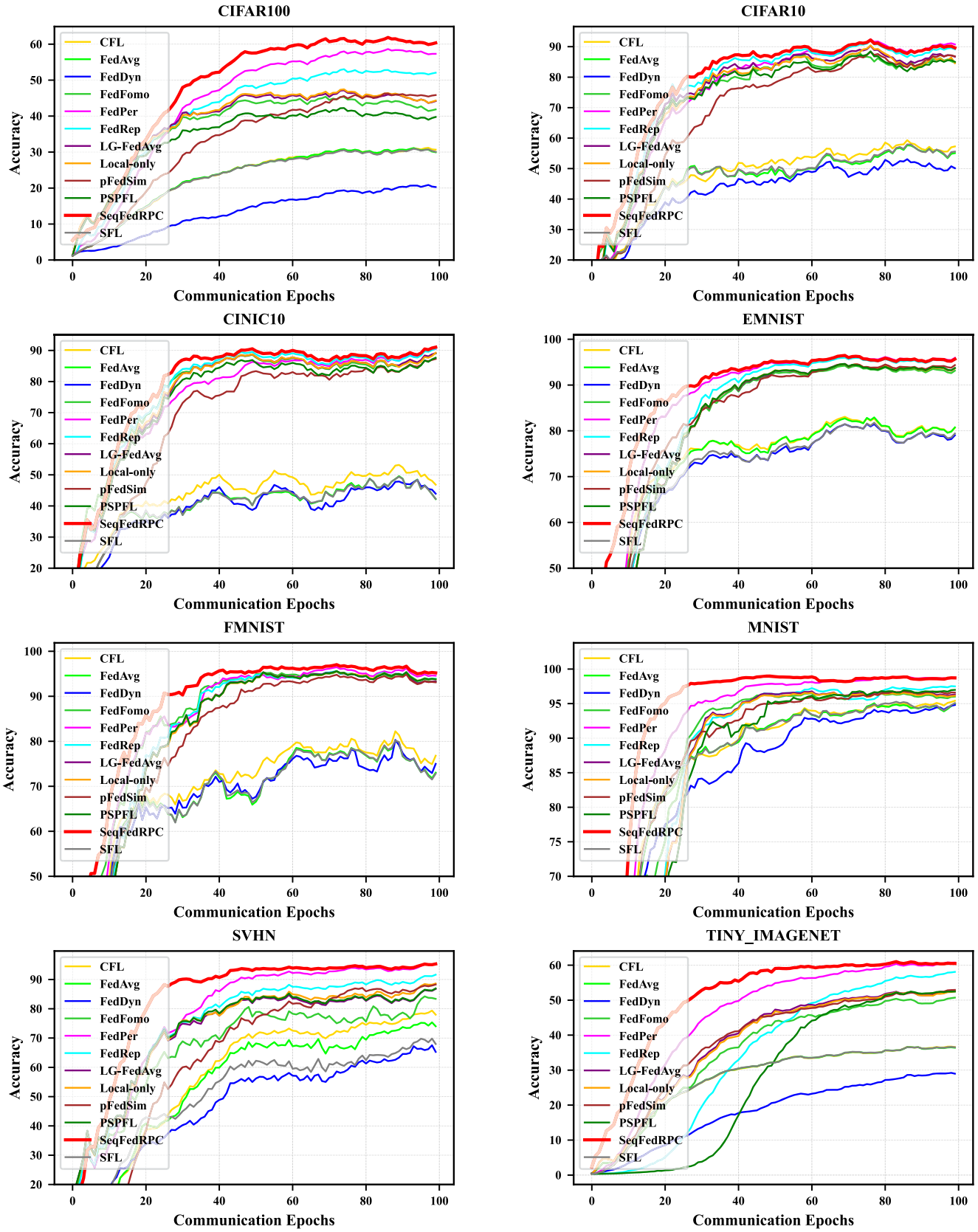


FIGURE 4 | Convergence comparison of SeqFedRPC and 11 solutions with $\alpha = 0.1$ on the VGG-11 model across eight datasets.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

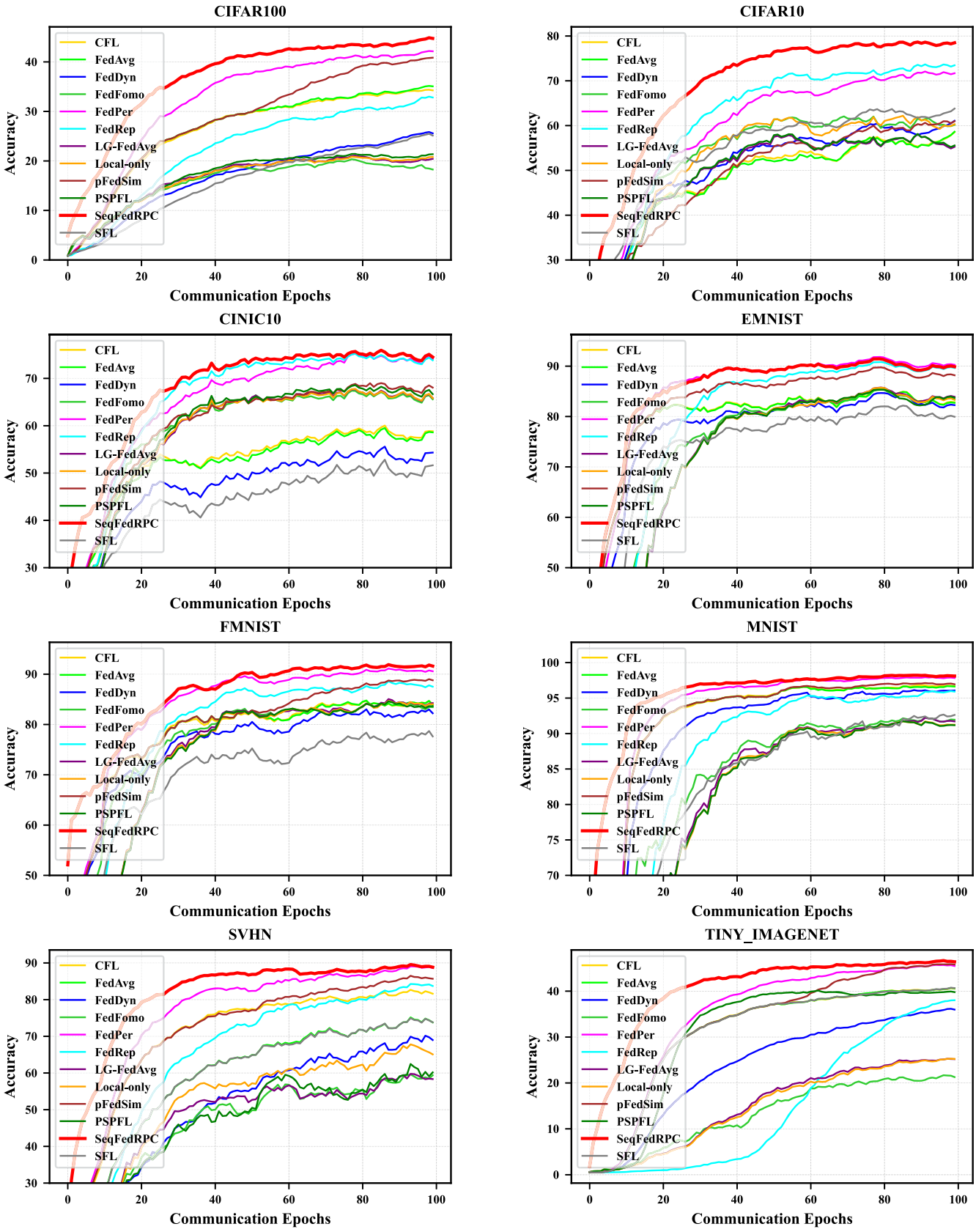


FIGURE 5 | Convergence comparison of `SeqFedRPC` and 11 solutions with $\alpha = 0.5$ on the VGG-11 model across eight datasets.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

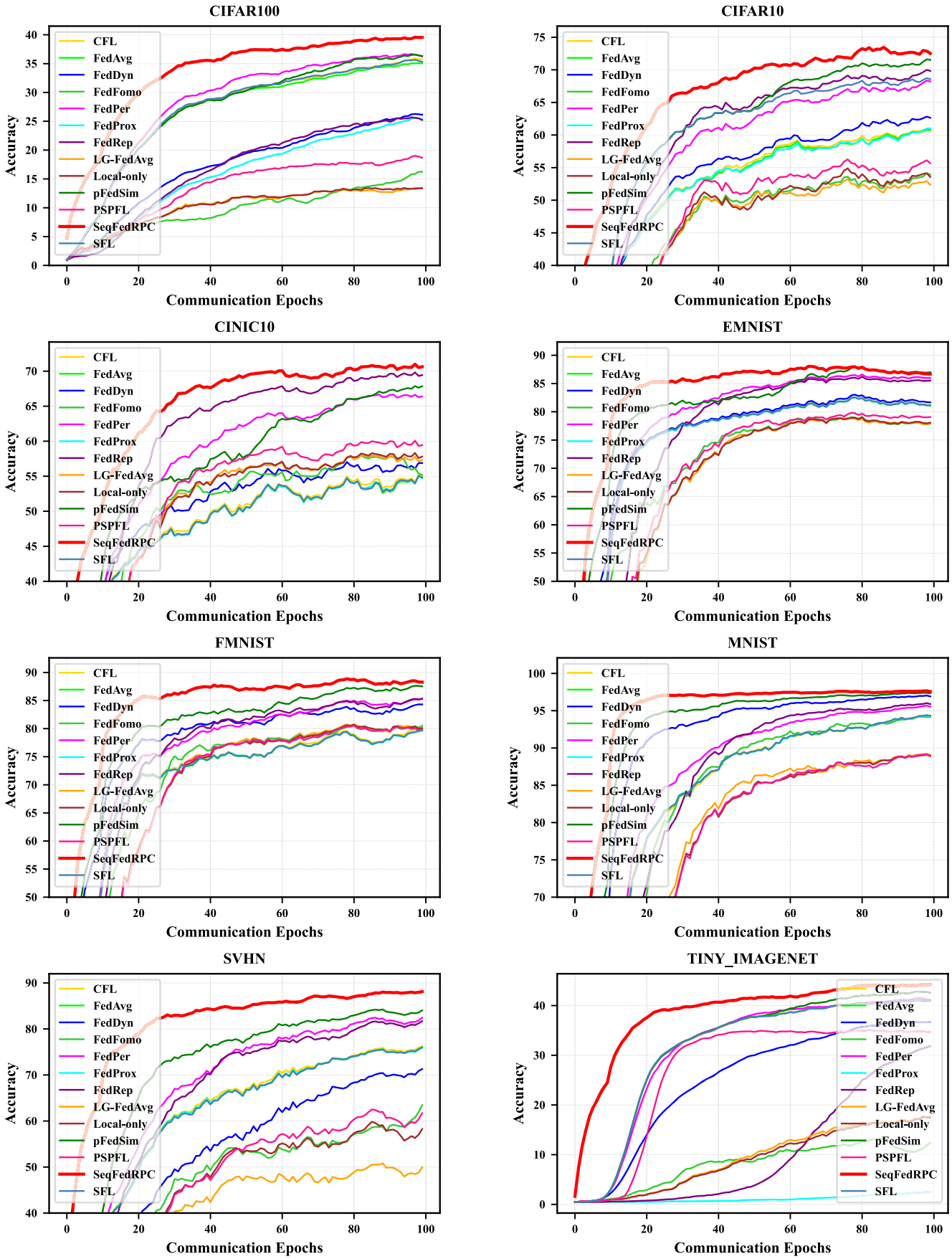


FIGURE 6 | Convergence comparison of `SeqFedRPC` and 11 solutions with $\alpha = 1.0$ on the VGG-11 model across eight datasets.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

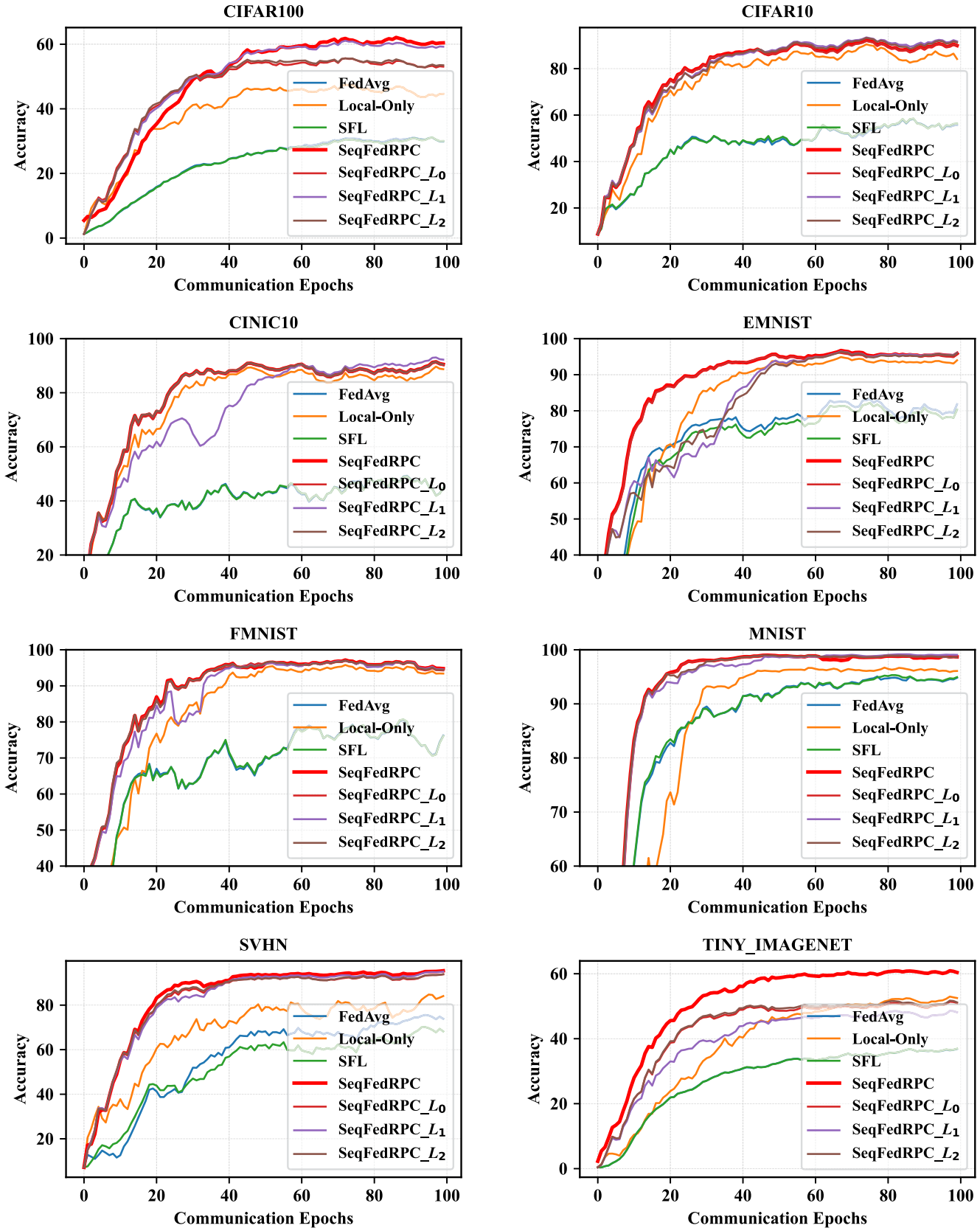


FIGURE 7 | Ablation results. Accuracy comparison of `SeqFedRPC`, variants and baselines (`FedAvg`, `SFL`, and `Local-only`) across eight datasets.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 7 | Performance (%) comparison of SeqFedRPC, variants and baselines across eight datasets.

Method	CIFAR10	CIFAR100	CINIC10	SVHN	EMNIST	FMNIST	MNIST	T.IN.
FedAvg	54.983	30.382	45.894	72.488	80.448	76.434	94.648	35.999
Local-only	85.523	45.269	86.500	80.469	93.582	94.376	96.264	51.551
SFL	55.557	29.983	45.701	65.842	79.591	76.724	94.779	36.063
SeqFedRPC_L_0	89.343	53.705	88.659	92.684	95.288	95.693	98.680	50.364
SeqFedRPC_L_1	**91.334**	59.488	**90.954**	93.718	**95.537**	95.673	**99.027**	47.596
SeqFedRPC_L_2	90.806	53.970	88.988	92.800	95.363	95.744	98.749	50.768
SeqFedRPC	89.528	**60.488**	88.885	**94.562**	95.513	**95.896**	98.673	**60.342**

Note: Best results are highlighted in bold, and second-best results in underline.

TABLE 8 | Shared and personalized parameter ratios under different α settings.

Dataset	$\alpha = 0.1$		$\alpha = 0.5$		$\alpha = 1.0$	
	Shared	Personalized	Shared	Personalized	Shared	Personalized
CIFAR10	0.9724	0.0276	0.9566	0.0434	0.7832	0.2168
CIFAR100	0.9522	0.0478	0.9481	0.0519	0.7944	0.2056
CINIC10	0.9787	0.0213	0.9593	0.0407	0.7875	0.2125
SVHN	0.9981	0.0019	0.9424	0.0576	0.7903	0.2097
EMNIST	0.9929	0.0071	0.9598	0.0402	0.7528	0.2472
FMNIST	0.9892	0.0108	0.9612	0.0388	0.7537	0.2463
MNIST	0.9265	0.0735	0.8392	0.1608	0.7572	0.2428
T.IN.	0.9027	0.0973	0.8254	0.1746	0.5781	0.4219

Tiny ImageNet. It is worth noting that even the variant using only the cross-entropy loss (SeqFedRPC_L_0) consistently outperforms the three baseline methods—FedAvg, SFL, and Local-only—demonstrating the effectiveness of our model decomposition-based SFL framework.

5.5 | Parameter Ratio Discussion

To investigate the impact of data heterogeneity on the division between shared and personalized parameters within the model, we analyze the proportions of parameters at different levels of Dirichlet distribution ($\alpha = 0.1,\ 0.5,\ 1.0$), as shown in Table 8. All hyperparameters are aligned with those used in the main experiments. Since our method adaptively customizes parameter partitioning for each individual client, the resulting proportions vary across the 100 clients. The values reported in Table 8 represent the average proportions of parameters in all clients during the first 30 communication rounds, at which point the partitioning process has largely stabilized and thus serves as a meaningful reference.

We note that each dataset possesses a smallest number of image categories under $\alpha = 0.1$ compare to $\alpha = 0.5$ and $\alpha = 1.0$, which results in a reduced requirement for personalized parameters. For example, our solution allocates 9.73% of personalized parameters with $\alpha = 0.1$ on the Tiny ImageNet, which is lower than 17.46% with $\alpha = 0.5$ and 42.19% with $\alpha = 1.0$. As α increases, user homogeneity also increases, and each user has more data categories. The regularized loss function of SeqFedRPC optimization compels models to select more personalized parameters to balance training data fitting with category generalization. Consequently, the need for personalized parameters rises due to the increased variety of data categories associated with each user. The overall trend emphasizes a growing need for customization in model parameters as user homogeneity increases. Across the datasets, CIFAR10, CINIC10, and SVHN exhibit high shared parameter ratios at $\alpha = 0.1$ but show significant declines as α increases, indicating a shift toward personalized parameters. In contrast, Tiny ImageNet highlight a stronger necessity for personalized parameters, especially evident at $\alpha = 1.0$ with 42.19%.

5.6 | Scalability

This subsection evaluates the performance of the proposed algorithm, SeqFedRPC, under varying client participation ratios $\beta = \{0.1,0.2,0.3,0.4,0.5\}$. The results are presented in Table 9, where the highest accuracy values are highlighted in bold and the second-best results are underlined. For ease of interpretation, we also include the standard deviation for each dataset.

As shown in Table 9, the accuracy of SeqFedRPC generally increases with a higher client participation rate, although the peak performance varies across datasets. For most datasets (except Tiny ImageNet), SeqFedRPC exhibits minimal performance fluctuations across different participation levels, with accuracy standard deviations not exceeding 1, indicating strong stability. For example, on the SVHN dataset, SeqFedRPC achieves its highest accuracy at $\beta = 0.1$, after which performance

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | Accuracy (%) of SeqFedRPC under varying client participation ratios.

Dataset	0.1	0.2	0.3	0.4	0.5	Std. dev.
CIFAR10	89.528	90.336	89.994	90.469	**90.947**	0.531
CIFAR100	53.487	**54.325**	53.710	53.063	53.192	0.499
CINIC10	88.885	89.175	89.592	**89.602**	89.414	0.305
SVHN	**94.562**	93.814	93.606	93.932	94.007	0.357
EMNIST	95.513	95.157	95.622	**95.879**	95.647	0.263
FMNIST	95.896	95.929	**96.617**	96.486	96.563	0.355
MNIST	98.673	**98.987**	98.965	98.916	98.862	0.126
T.IN.	**55.018**	50.908	50.512	50.336	50.893	1.963

gradually declines. On CINIC10, the accuracy steadily improves from 88.885% to 89.602%, followed by a slight drop to 89.414%. In contrast, the Tiny ImageNet dataset shows relatively larger fluctuations, reaching its highest accuracy of 55.018% at $\beta = 0.1$, with a standard deviation of 1.963, suggesting higher sensitivity to changes in client participation rate.

6 | Conclusions

We propose SeqFedRPC, a novel two-stage framework designed to effectively address data heterogeneity in personalized federated learning by personalizing only about 10% of model parameters (at $\alpha = 0.1$). The framework combines regularization-based sparsification with a gradient amplification strategy and uses K-means clustering to automatically distinguish between shared and personalized parameters. Parameters with the largest gradient magnitudes are retained for local training as personalized components. Extensive experiments on eight benchmark datasets demonstrate that SeqFedRPC consistently outperforms eight state-of-the-art methods in accuracy. Ablation studies further validate the effectiveness of our regularization-based selective parameter aggregation strategy. These results establish SeqFedRPC as a practical and advanced solution for real-world sequential personalized federated learning applications.

7 | Limitations and Future Work

Despite the success of SeqFedRPC in personalized SFL, several limitations merit attention. The framework relies on K-Means clustering, which may not efficiently handle diverse gradient distributions. Scalability also remains a concern, as clustering and parameter management introduce nontrivial computational overhead, particularly in large-scale deployments. Moreover, our evaluation is currently limited to specific network architectures, including LeNet-5, ResNet-18, and VGG-11. Future research should focus on two key directions: (1) developing adaptive clustering techniques and dynamic parameter selection strategies to improve generality and flexibility; and (2) expanding evaluations to more diverse model architectures to enhance scalability and establishing theoretical guarantees of convergence.

Acknowledgments

The research is supported by Beijing University of Posts and Telecommunications-China Mobile Communications Group Co. Ltd. Joint Institute, the National Natural Science Foundation of China under Grant Nos. 62302400 and the Doctoral Research Project of Research Institute for Digital Economy and Interdisciplinary Sciences under Grant 222510004005040043.

Funding

This work was supported in part by Smart Gird-National Science and Technology Major Project under Grant 2025ZD0804600, and in part by the National Natural Science Foundation of China under Grant 62302400, 62471066, 62472240, 62394322, U22B2031, and in part by the Fundamental Research Funds for the Beijing University of Posts and Telecommunications under Grant 2025AI4S08, and in part by the Doctoral Research Project of Research Institute for Digital Economy and Interdisciplinary Sciences under Grant 222510001002020036.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

1. P. Kairouz, H. B. McMahan, B. Avent, et al., "Advances and Open Problems in Federated Learning," *Foundations and Trends in Machine Learning* 14, no. 1–2 (2021): 1–210.

2. J. Kaplan, S. McCandlish, T. Henighan, et al., "Scaling Laws for Neural Language Models," (2020), arXiv preprint arXiv:2001.08361.

3. J. Hoffmann, S. Borgeaud, A. Mensch, et al., "Training Compute-Optimal Large Language Models," *in Proceedings of the 36th International Conference on Neural Information Processing Systems* (NeurIPS) (2022), 30016–30030.

4. W. Kuang, B. Qian, Z. Li, et al., "Federatedscope-Llm: A Comprehensive Package for Fine-Tuning Large Language Models in Federated Learning," *in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (ACM SIGKDD) (2024), 5260–5271.

5. N. Wang, Y. Deng, W. Feng, S. Fan, J. Yin, and S. K. Ng, "One-Shot Sequential Federated Learning for Non-IID Data by Enhancing Local Model Diversity," *in Proceedings of the 32nd ACM International Conference on Multimedia* (ACM MM) (2024), 5201–5210.

6. R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of Collective Communication Operations in MPICH," *International Journal of High Performance Computing Applications* 19, no. 1 (2005): 49–66.

7. M. Yu, Y. Tian, B. Ji, C. Wu, H. Rajan, and J. Liu, "Gadget: Online Resource Optimization for Scheduling Ring-All-Reduce Learning Jobs,"

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

IEEE INFOCOM 2022-IEEE Conference on Computer Communications (IEEE) (2022), 1569–1578.

8. D. De Sensi, T. Bonato, D. Saam, and T. Hoefler, "Swing: Short-Cutting Rings for Higher Bandwidth Allreduce," *in 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (2024), 1445–1462.

9. M. Ye, X. Fang, B. Du, P. C. Yuen, and D. Tao, "Heterogeneous Federated Learning: State-Of-The-Art and Research Challenges," *ACM Computing Surveys* 56, no. 3 (2023): 1–44.

10. Z. Zhu, J. Hong, and J. Zhou, "Data-Free Knowledge Distillation for Heterogeneous Federated Learning," in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139, ed. M. Meila and T. Zhang (PMLR, 2021), 12878–12889.

11. D. Yao, W. Pan, Y. Dai, et al., "FedGKD: Towards Heterogeneous Federated Learning via Global Knowledge Distillation," *IEEE Transactions on Computers* 73, no. 1 (2024): 3–17.

12. A. Ghosh, J. Chung, D. Yin, and K. Ramchandran, "An Efficient Framework for Clustered Federated Learning," *Advances in Neural Information Processing Systems* 33 (2020): 19586–19597.

13. L. Yang, J. Huang, W. Lin, and J. Cao, "Personalized Federated Learning on Non-IID Data via Group-Based Meta-Learning," *ACM Transactions on Knowledge Discovery From Data* 17, no. 4 (2023): 1–20.

14. L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting Shared Representations for Personalized Federated Learning," *in International Conference on Machine Learning (PMLR)* (2021), 2089–2099.

15. J. Oh, S. Kim, and S. Yun, "FedBABU: Toward Enhanced Representation for Federated Image Classification," *in 10th International Conference on Learning Representations, ICLR* (2022).

16. J. Xu, X. Tong, and S. L. Huang, "Personalized Federated Learning With Feature Alignment and Classifier Collaboration," *in the Eleventh International Conference on Learning Representations (ICLR)* (2023).

17. X. Yang, W. Huang, and M. Ye, "Dynamic Personalized Federated Learning With Adaptive Differential Privacy," *in Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)* (2023).

18. H. Zhou, M. Li, P. Sun, B. Guo, and Z. Yu, "Accelerating Federated Learning via Parameter Selection and Pre-Synchronization in Mobile Edge-Cloud Networks," *IEEE Transactions on Mobile Computing* 23, no. 11 (2024): 10313–10328, https://doi.org/10.1109/TMC.2024.3376636.

19. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE* 86, no. 11 (1998): 2278–2324.

20. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), 770–778.

21. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *in 3rd International Conference on Learning Representations (ICLR 2015)* (Computational and Biological Learning Society: 2015).

22. Z. Yang, Y. Zhang, Y. Zheng, et al., "FedFed: Feature Distillation Against Data Heterogeneity in Federated Learning," in *Advances in Neural Information Processing Systems*, vol. 36, ed. A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (NeurIPS, Curran Associates Inc, 2023), 60397–60428.

23. R. Zhang, F. Liu, J. Liu, et al., "CPPer-FL: Clustered Parallel Training for Efficient Personalized Federated Learning," *IEEE Transactions on Mobile Computing* 23 (2024): 9424–9436.

24. J. Tan, Y. Zhou, G. Liu, J. H. Wang, and S. Yu, "pFedSim: Similarity-Aware Model Aggregation Towards Personalized Federated Learning," (2023), arXiv preprint arXiv:2305.15706.

25. Y. Li and X. Lyu, "Convergence Analysis of Sequential Federated Learning on Heterogeneous Data," *Advances in Neural Information Processing Systems* 36 (2023): 56700–56755.

26. J. Lee, J. Oh, S. Lim, S. Y. Yun, and J. G. Lee, "Tornadoaggregate: Accurate and Scalable Federated Learning via the Ring-Based Architecture," (2020), arXiv preprint arXiv:2012.03214.

27. R. Zaccone, A. Rizzardi, D. Caldarola, M. Ciccone, and B. Caputo, "Speeding Up Heterogeneous Federated Learning With Sequentially Trained Superclients," *in 2022 26th International Conference on Pattern Recognition (ICPR)* (IEEE: 2022), 3376–3382.

28. M. Zeng, X. Wang, W. Pan, and P. Zhou, "Heterogeneous Training Intensity for Federated Learning: A Deep Reinforcement Learning Approach," *IEEE Transactions on Network Science and Engineering* 10, no. 2 (2022): 990–1002.

29. M. Kamp, J. Fischer, and J. Vreeken, "Federated Learning From Small Datasets," *in Eleventh International Conference on Learning Representations* (OpenReview. net) (2023).

30. Y. J. Cho, P. Sharma, G. Joshi, Z. Xu, S. Kale, and T. Zhang, "On the Convergence of Federated Averaging With Cyclic Client Participation," *in International Conference on Machine Learning* (PMLR) (2023), 5677–5721.

31. X. Yan, S. Zuo, R. Fan, et al., "Sequential Federated Learning in Hierarchical Architecture on Non-IID Datasets," (2024), arXiv preprint arXiv:2408.09762.

32. A. Krizhevsky, "Learning Multiple Layers of Features From Tiny Images," Master's thesis, University of Tront (2009).

33. R. Tamirisa, C. Xie, W. Bao, A. Zhou, R. Arel, and A. Shamsian, "Fedselect: Personalized Federated Learning With Customized Selection of Parameters for Fine-Tuning," *in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (CVPR) (2024), 23985–23994.

34. B. McMahan, E. Moore, D. Ramage, S. Hampson, and y B A. Arcas, "Communication-Efficient Learning of Deep Networks From Decentralized Data," *in Artificial Intelligence and Statistics* (PMLR) (2017), 1273–1282.

35. F. Sattler, K. R. Müller, and W. Samek, "Clustered Federated Learning: Model-Agnostic Distributed Multitask Optimization Under Privacy Constraints," *IEEE Transactions on Neural Networks and Learning Systems* 32, no. 8 (2020): 3710–3722.

36. D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. Whatmough, and V. Saligrama, "Federated Learning Based on Dynamic Regularization," *in International Conference on Learning Representations* (ICLR) (2021).

37. M. Zhang, K. Sapra, S. Fidler, S. Yeung, and J. M. Alvarez, "Personalized Federated Learning With First Order Model Optimization," *in International Conference on Learning Representations* (ICLR) (2021).

38. M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated Learning With Personalization Layers," (2019), arXiv preprint arXiv:1912.00818.

39. P. P. Liang, T. Liu, L. Ziyin, et al., "Think Locally, Act Globally: Federated Learning With Local and Global Representations," (2020), arXiv preprint arXiv:2001.01523.

Appendix A

Model Architecture

Our experiments primarily utilize LeNet-5 [19], VGG-11 [21], and ResNet-18 [20] as the backbone models. The detailed architectures are presented in Tables A1–A3, respectively, with layer representations following the PyTorch-style notation.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE A1 | LeNet-5 architecture. Conv2D() consists of a 2D convolution layer, batch normalization layer, and ReLU activation executed sequentially.

Component	Layer
Feature extractor	Conv2D ($in = 3$, $out = 6$, $kernel = 5$, $stride = 1$, $pad = 0$)
	Conv2D ($in = 6$, $out = 16$, $kernel = 5$, $stride = 1$, $pad = 0$)
	MaxPool2D ($kernel = 2$, $stride = 2$)
	Flatten()
	FC ($out = 120$)
	FC ($out = 84$)
Classifier	FC ($out = num_classes$)

Note: in, *out*, *kernel*, *stride*, and *pad* denote the input channels, output channels, kernel size, stride, and padding size, respectively. MaxPool2D() represents a 2D max pooling layer, Flatten() reshapes the input from 2D to 1D, and FC() denotes a fully connected layer where *out* indicates the output features.

TABLE A2 | VGG-11 architecture. Conv2D() consists of a 2D convolution layer, batch normalization layer, and ReLU activation, which is executed sequentially.

Component	Layer
Feature extractor	Conv2D ($in = 3$, $out = 64$, $kernel = 3$, $stride = 1$, $pad = 1$)
	Conv2D ($in = 64$, $out = 128$, $kernel = 3$, $stride = 1$, $pad = 1$)
	Conv2D ($in = 128$, $out = 256$, $kernel = 3$, $stride = 1$, $pad = 1$)
	Conv2D ($in = 256$, $out = 256$, $kernel = 3$, $stride = 1$, $pad = 1$)
	Conv2D ($in = 256$, $out = 512$, $kernel = 3$, $stride = 1$, $pad = 1$)
	Conv2D ($in = 512$, $out = 512$, $kernel = 3$, $stride = 1$, $pad = 1$)
	Conv2D ($in = 512$, $out = 512$, $kernel = 3$, $stride = 1$, $pad = 1$)
	Conv2D ($in = 512$, $out = 512$, $kernel = 3$, $stride = 1$, $pad = 1$)
	MaxPool2D ($kernel = 2$, $stride = 2$)
	Flatten()
	FC ($out = 4096$)
	FC ($out = 4096$)
Classifier	FC ($out = num_classes$)

Note: The *in, out, kernel, stride, pad* represent the input channel, output channel, kernel size, convolution kernel step size moving, and padding size, respectively; MaxPool2D() is a max pooling layer for 2D input; Flatten() is for reshaping input from 2D to 1D; FC() is a fully connected layer; out means the number of features output has.

TABLE A3 | ResNet-18 architecture. Conv2D() represents a 2D convolution layer followed by batch normalization and ReLU activation.

Component	Layer
Feature extractor	Conv2D ($in = 3$, $out = 64$, $kernel = 7$, $stride = 2$, $pad = 3$)
	MaxPool2D ($kernel = 3$, $stride = 2$, $pad = 1$)
	Residual Block 1: Conv2D ($in = 64$, $out = 64$, $kernel = 3$, $stride = 1$, $pad = 1$) $\times 2$
	Residual Block 2: Conv2D ($in = 64$, $out = 128$, $kernel = 3$, $stride = 2$, $pad = 1$) $\times 2$
	Residual Block 3: Conv2D ($in = 128$, $out = 256$, $kernel = 3$, $stride = 2$, $pad = 1$) $\times 2$
	Residual Block 4: Conv2D ($in = 256$, $out = 512$, $kernel = 3$, $stride = 2$, $pad = 1$) $\times 2$
	Global Average Pooling (GAP)
	Flatten()
	FC ($out = 1000$)
Classifier	FC ($out = num_classes$)

Note: Each residual block consists of two sequential convolution layers with skip connections. The *in*, *out*, *kernel*, *stride*, and *pad* denote the input channels, output channels, kernel size, stride, and padding size, respectively. Global Average Pooling (GAP) reduces the spatial dimensions to 1. FC() represents a fully connected layer, where *out* is the number of output features.

21613915, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/ett.70477 by Xingyan Chen - Southwestern University Of Finance And Economics , Wiley Online Library on [03/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License